

# Documento de Prueba Unitaria — Asignación Automática de Slots (xUnit)

**Proyecto:** ANPR-VISION (Backend)

**Módulo:** Business → `VehicleBusiness.RegisterVehicleWithSlotAsync`

**Versión del documento:** 1.0

**Autor:** Karol Natalia Osorio Poveda

**Fecha de ejecución:** 08/09/2025

**Duración estimada:** 10–15 min

**Tipo de prueba:** Unitaria

**Herramientas:** .NET SDK (9), xUnit, Moq, FluentAssertions, Test Explorer

**Repositorio/Ubicación de pruebas:**

`tests/AnprVision.Business.Tests/VehicleBusiness_RegisterSlot_Tests.cs`

## 1. Resumen ejecutivo

Validar la lógica de **asignación automática de espacios (slots)** cuando se registra un vehículo en el parqueadero. El flujo:

1. Obtener vehículo.
2. Consultar **sectores compatibles** por tipo de vehículo.
3. Filtrar **slots disponibles** (no ocupados y `IsAvailable = true`).
4. Elegir un slot y **marcarlo ocupado** (`IsAvailable = false`).
5. Crear `RegisteredVehicles` con `VehicleId`, `SlotsId` y `EntryDate`.

**Resultado esperado global:**

- Se asigna un slot válido cuando existe disponibilidad.
- Se rechaza si el vehículo no existe o si **no hay slots disponibles**.

## 2. Reglas/Req. de negocio cubiertos

- Solo se consideran **sectores compatibles** con el `TypeVehicleId` del vehículo.
- Un slot es asignable si está **disponible** ( `IsAvailable = true` ) y **no** tiene un `RegisteredVehicles` **activo**.
- Al asignar, el slot debe quedar **ocupado** ( `IsAvailable = false` ) y debe crearse el `RegisteredVehicles` CON `EntryDate =.UtcNow` .
- Si el vehículo **no existe**, se detiene el proceso.
- Si **no hay slots** disponibles, el proceso falla con un mensaje coherente.

## 3. Entorno y dependencias

- **Entorno:** Local Dev.
- **Dependencias moqueadas:**
  - `IVehicleData` → `GetById(vehicleId)` .
  - `ISectorsData` → `GetSectorsByVehicleTypeAsync(typeId)` .
  - `IRegisteredVehiclesData` → `AnyActiveRegisteredVehicleInSlotAsync(slotId)` ,  
`Save(RegisteredVehicles)` .
  - `ISlotsData` → `Update(Slots)` (marcar ocupado).
  - `IMapper` → configuración mínima (no usado por el método, requerido por la clase base).

**SUT:** `VehicleBusiness`

**Método:** `Task<RegisteredVehicles> RegisterVehicleWithSlotAsync(int vehicleId)`

**Efectos esperados:** actualización del slot y alta de `RegisteredVehicles` con datos correctos.

## 4. Datos de entrada (mock)

- **Vehículos:**
  - Válido: { `Id = 1`, `TypeVehicleId = 10`, `Plate = "AAA111"` }
  - Inexistente: `Id = 999`
  - Válido (sin slots): { `Id = 2`, `TypeVehicleId = 20`, `Plate = "BBB222"` }
- **Slots/Sectores:**
  - Slot válido: { `Id = 7/8`, `IsAvailable = true/false` } dentro de un  
`Sectors { Id = ..., Slots = [...] }`

- **Ocupación:** controlada por `AnyActiveRegisteredVehicleInSlotAsync` (true/false).

## 5. Casos de prueba

### CT-01 — Caso válido: asigna slot y marca no disponible

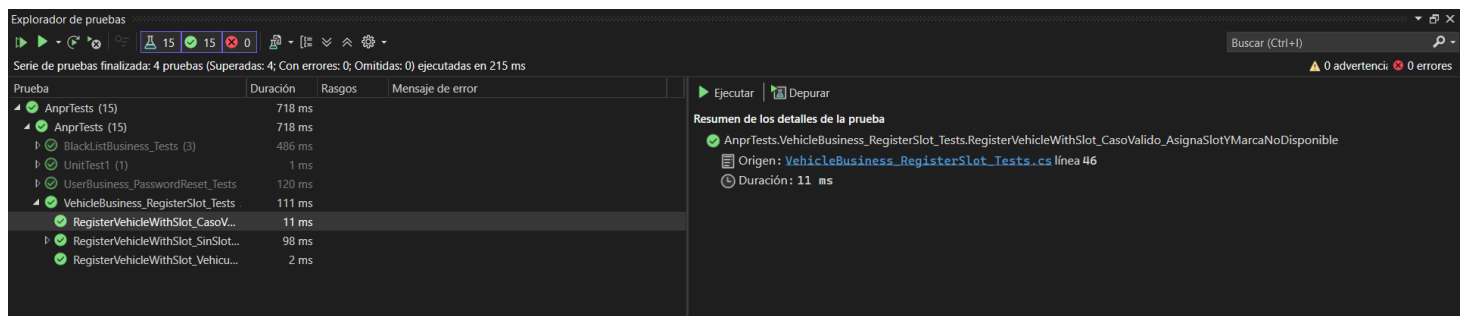
**Precondición:** Vehículo `Id=1` existe; sector compatible con **1 slot disponible**; no hay activo en ese slot.

**Pasos:** Ejecutar `RegisterVehicleWithSlotAsync(1)` .

**Assert:**

- `Slots.Update` CON `IsAvailable = false` .
- `RegisteredVehicles.Save` CON `VehicleId = 1` , `SlotsId = 7` , `EntryDate != default` .
- Retorna entidad con `Id` asignado (simulado 123 ) .

**Evidencia:**



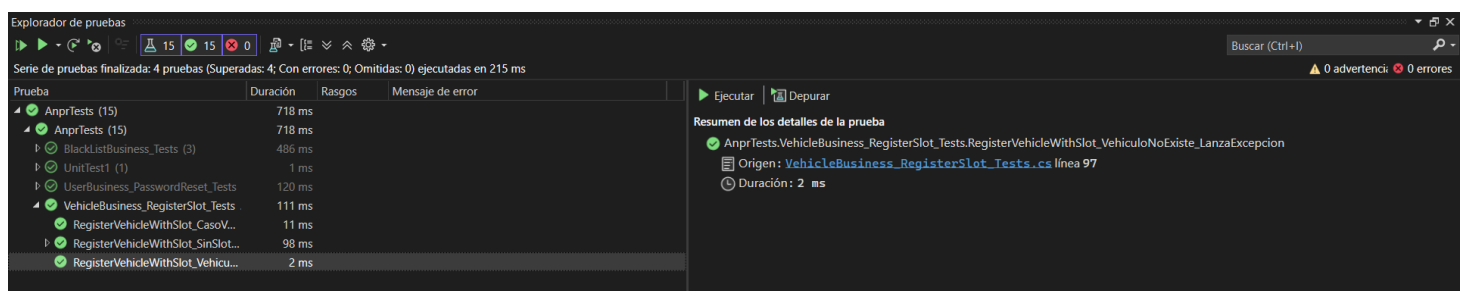
### CT-02 — Vehículo no existe → excepción

**Precondición:** `GetById(999)` → `null` .

**Pasos:** Ejecutar `RegisterVehicleWithSlotAsync(999)` .

**Assert:** Excepción con mensaje “Vehículo no encontrado.”

**Evidencia:**



## CT-03 — Sin slots disponibles → excepción

**Precondición:** Vehículo Id=2 existe; sector compatible con **1 slot** pero:

- **Caso A:** IsAvailable = false (no disponible).
- **Caso B:** IsAvailable = true pero AnyActiveRegisteredVehicleInSlotAsync(...) = true (ocupado).

**Pasos:** Ejecutar RegisterVehicleWithSlotAsync(2) para cada combinación.

**Assert:** Excepción con mensaje “No hay slots disponibles...”, **sin** Slots.Update y **sin** RegisteredVehicles.Save .

**Evidencia:**

Explorador de pruebas

Serie de pruebas finalizada: 4 pruebas (Superadas: 4; Con errores: 0; Omitidas: 0) ejecutadas en 215 ms

| Prueba                             | Duración | Rasgos | Mensaje de error |
|------------------------------------|----------|--------|------------------|
| AnprTests (15)                     | 718 ms   |        |                  |
| AnprTests (15)                     | 718 ms   |        |                  |
| BlackListBusiness_Tests (3)        | 486 ms   |        |                  |
| UnitTest1 (1)                      | 1 ms     |        |                  |
| UserBusiness_PasswordReset_Tests   | 120 ms   |        |                  |
| VehicleBusiness_RegisterSlot_Tests | 111 ms   |        |                  |
| RegisterVehicleWithSlot_CasoV...   | 11 ms    |        |                  |
| RegisterVehicleWithSlot_SinSlot... | 98 ms    |        |                  |
| RegisterVehicleWithSlot_SinSL...   | 96 ms    |        |                  |
| RegisterVehicleWithSlot_SinSL...   | 2 ms     |        |                  |
| RegisterVehicleWithSlot_Vehicu...  | 2 ms     |        |                  |

Ejecutar Depurar

Resumen del grupo

AnprTests.VehicleBusiness\_RegisterSlot\_Tests.RegisterVehicleWithSlot\_SinSlotsDisponibles\_LanzaExcepcion

Pruebas en grupo: 2

Origen: VehicleBusiness\_RegisterSlot\_Tests.cs línea 115

Duración total: 98 ms

Salidas

2 Correcta

## 6. Resultado de ejecución

**Resultado:** 4/4 pruebas superadas (0 fallidas).

**Evidencia:**

Explorador de pruebas

Serie de pruebas finalizada: 4 pruebas (Superadas: 4; Con errores: 0; Omitidas: 0) ejecutadas en 215 ms

| Prueba                             | Duración | Rasgos | Mensaje de error |
|------------------------------------|----------|--------|------------------|
| AnprTests (15)                     | 718 ms   |        |                  |
| AnprTests (15)                     | 718 ms   |        |                  |
| BlackListBusiness_Tests (3)        | 486 ms   |        |                  |
| UnitTest1 (1)                      | 1 ms     |        |                  |
| UserBusiness_PasswordReset_Tests   | 120 ms   |        |                  |
| VehicleBusiness_RegisterSlot_Tests | 111 ms   |        |                  |
| RegisterVehicleWithSlot_CasoV...   | 11 ms    |        |                  |
| RegisterVehicleWithSlot_SinSlot... | 98 ms    |        |                  |
| RegisterVehicleWithSlot_SinSL...   | 96 ms    |        |                  |
| RegisterVehicleWithSlot_SinSL...   | 2 ms     |        |                  |
| RegisterVehicleWithSlot_Vehicu...  | 2 ms     |        |                  |

Ejecutar Depurar

Resumen del grupo

VehicleBusiness\_RegisterSlot\_Tests

Pruebas en grupo: 4

Duración total: 111 ms

Salidas

4 Correcta

## 7. Criterios de aceptación

- Se usan **sectores compatibles** y se filtran slots por **disponibilidad real** (no ocupado, `IsAvailable = true` ).
- En caso válido, el slot queda **ocupado** y se persiste un `RegisteredVehicles` correcto.
- Se detiene con mensajes coherentes si el vehículo no existe o no hay slots disponibles.

## 8. Comentario personal

- Las pruebas cubren los **tres escenarios críticos** del flujo y respalda la regla de negocio de **disponibilidad real**.
- Se recomienda endurecer la **sección atómica** de actualización de slot para evitar asignaciones dobles en escenarios concurrentes.